

UNIVERSIDAD DE LA HABANA

Facultad de Matemática y Computación



Proyecto Final de Inteligencia Artificial

Segmentación Óptima de Contenido

Autor: Lidier Robaina Caraballo

Grupo: C-412

mayo de 2026

Índice general

1. Introducción	1
2. Diseño e Implementación	2
2.1. Modelado formal	2
2.2. Solución exacta con programación dinámica	3
2.3. Metaheurísticas	3
2.3.1. Búsqueda local	3
2.3.2. Algoritmo genético	4
2.4. Integración del LLM	4
2.5. Dataset	4
3. Resultados y Análisis	5
3.1. Experimento 1: Ajuste de la función objetivo	5
3.1.1. Objetivo	5
3.1.2. Función objetivo propuesta	5
3.1.3. Metodología	6
3.1.4. Resultados esperados	6
3.1.5. Resultados obtenidos	6
3.1.6. Selección final de parámetros	8
3.2. Experimento 2: Ajuste de las metaheurísticas	9
3.2.1. Objetivo	9
3.2.2. Metodología	9
3.2.3. Ascenso de colina (Hill Climbing)	9
3.2.4. Recocido simulado (Simulated Annealing)	10
3.2.5. Algoritmo genético (Genetic Algorithm)	11
3.2.6. Enjambre de partículas (Particle Swarm Optimization)	13
3.3. Experimento 3: Evaluación de las metaheurísticas	14
3.3.1. Objetivo	14
3.3.2. Metodología	14
3.3.3. Resultados esperados	14
3.3.4. Resultados obtenidos	15
3.3.5. Conclusiones del experimento 3	15
4. Conclusiones	17

Capítulo 1

Introducción

La segmentación de contenido es una tarea fundamental en la organización automática de información: dividir una secuencia de elementos (párrafos, oraciones, fragmentos de vídeo) en bloques contiguos que presenten coherencia temática interna. Una segmentación de calidad permite mejorar la navegación en documentos extensos, la generación de resúmenes y la extracción de información. Tradicionalmente, la tarea se ha abordado con métodos basados en similitud léxica o modelos probabilísticos; sin embargo, la aparición de grandes modelos de lenguaje (LLM) ofrece la posibilidad de evaluar la coherencia semántica de forma mucho más precisa, actuando como un oráculo que puntúa la cohesión de un segmento.

En este proyecto se trata la segmentación como un problema de optimización combinatoria: encontrar la partición de una secuencia que maximiza una función de coherencia agregada, evaluada mediante un LLM. Este enfoque permite estudiar cómo distintas estrategias algorítmicas pueden encontrar soluciones de alta calidad gestionando eficientemente un recurso costoso y limitado.

Se plantean los siguientes objetivos:

- Formalizar el problema de segmentación óptima basada en coherencia.
- Implementar una solución exacta con programación dinámica que sirva como referencia (*baseline*) para instancias reducidas.
- Desarrollar metaheurísticas que exploren el espacio de segmentaciones utilizando al LLM como función de coste.
- Diseñar un mecanismo de interacción con el LLM que incluya un *prompt* robusto, caché de evaluaciones y estrategias para reducir el número de llamadas.
- Construir un conjunto de instancias de prueba que permitan ajustar y analizar el comportamiento de los algoritmos.
- Comparar experimentalmente las configuraciones propuestas en términos de calidad de la solución, número de consultas al LLM y tiempo de ejecución.

Capítulo 2

Diseño e Implementación

2.1 Modelado formal

Sea la entrada del problema $S = (e_1, e_2, \dots, e_N)$, una secuencia finita de N elementos textuales. Se desea obtener una segmentación que maximice la suma de las coherencias individuales, sujeta a que los segmentos sean disjuntos, cubran completamente S y respeten el orden original.

Con este fin, se define una segmentación como un vector binario $\mathbf{x} \in \{0, 1\}^{N-1}$, donde $x_i = 1$ indica que existe un corte entre e_i y e_{i+1} . El número de segmentos es $k = 1 + \sum x_i$. Sea la función objetivo:

$$f(\mathbf{x}) = \sum_{j=1}^k \phi(s_j)$$

donde $\phi(s)$ es la puntuación de coherencia del segmento s devuelta por el LLM, la salida del problema es la segmentación $\mathbf{x}$ que maximiza f .

Sin embargo, por la naturaleza del dominio del problema y la función objetivo anterior, la solución para cualquier instancia del problema sería la solución degenerada de cortar cada elemento. Por tanto, para capturar el coste de generar más segmentos y penalizar segmentaciones excesivamente fragmentadas, la función objetivo final incorpora un término de penalización (α) sobre el número de segmentos:

$$f(\mathbf{x}) = \sum_{j=1}^k \phi(s_j) - \alpha \cdot k$$

Adicionalmente, los segmentos de longitud uno (en lo adelante, segmentos unitarios) reciben una puntuación fija ϕ_u . Esta parametrización permite ajustar el equilibrio entre coherencia intrasegmental y parsimonia de la partición.

El problema es de naturaleza combinatoria: si se permite cualquier número de puntos de corte, el espacio de soluciones crece exponencialmente (2^{N-1}). Por tanto, se investiga

una solución exacta de referencia para instancias pequeñas y metaheurísticas capaces de aproximar soluciones buenas limitando el número de evaluaciones del LLM.

2.2 Solución exacta con programación dinámica

El módulo `dp.py` implementa un algoritmo de programación dinámica (DP) que calcula la segmentación óptima. Se define $dp[i]$ como la máxima puntuación alcanzable para los primeros i elementos. La recurrencia es:

$$dp[i] = \max_{0 \leq j < i} (dp[j] + \phi(e_{j+1} \dots e_i) - \alpha)$$

con $dp[0] = 0$. La solución se reconstruye almacenando los puntos de corte óptimos.

El algoritmo es de complejidad $O(N^2 \cdot C)$, donde C es el coste de una evaluación de coherencia. Se utiliza para ajustar los parámetros de la función objetivo, α y ϕ_u , cuando las instancias son lo bastante pequeñas para evaluar exhaustivamente.

2.3 Metaheurísticas

El archivo `metaheuristics.py` define una jerarquía clara de clases que facilita la extensión y el análisis comparativo.

La clase abstracta `Metaheuristic` obliga a todos los algoritmos a implementar un método `solve` que retorna la mejor solución, su fitness y un diccionario de estado.

A partir de ella se definen dos plantillas:

- `SingleSolutionMetaheuristic`: para algoritmos de búsqueda de trayectoria que trabajan sobre una única solución en cada iteración.
- `PopulationMetaheuristic`: para algoritmos evolutivos que manejan una población completa.

Ambas plantillas estandarizan el seguimiento de métricas como número de evaluaciones, iteraciones/generaciones y mejor fitness encontrado.

2.3.1. Búsqueda local

Las implementaciones concretas de búsqueda local heredan de `SingleSolutionMetaheuristic`:

- `HillClimbing`: genera un vecino mediante `problem.get_neighbor`, acepta únicamente mejoras y detiene la búsqueda tras un número fijo de iteraciones sin mejora.

- **SimulatedAnnealing**: también usa `problem.get_neighbor`, pero acepta soluciones peores según una probabilidad que depende de la temperatura. La temperatura se enfría geométricamente para equilibrar explotación y exploración.

Ambos algoritmos comparten la misma representación de solución binaria y delegan en `ContentSegmentationProblem` la generación de vecinos y la evaluación.

2.3.2. Algoritmo genético

La clase `GeneticAlgorithm` hereda de `PopulationMetaheuristic` y define:

- inicialización de población con soluciones aleatorias,
- selección por torneo binario,
- cruce de un punto en el vector binario de cortes,
- mutación bit-flip con baja probabilidad,
- reemplazo con elitismo para mantener los mejores individuos.

Esta implementación aprovecha la misma representación binaria y los operadores del problema para que la evolución sea compatible con la función de evaluación basada en LLM.

2.4 Integración del LLM

La evaluación de coherencia está centralizada en `llm.py`. Para cada segmento se realiza lo siguiente:

- segmentos unitarios reciben una coherencia fija (valor `unit_coherence`), evitando llamadas innecesarias al modelo;
- para segmentos mayores se construye un prompt que solicita una puntuación entre 0 y 1;
- la respuesta se parsea a número decimal y se guarda en caché en `data/llm_cache.json`;
- ante errores de API (por ejemplo 429) se aplica reintento exponencial y rotación de credenciales.

El uso de caché es crítico para amortiguar el coste de las evaluaciones repetidas en las metaheurísticas y para permitir comparaciones fiables entre configuraciones.

2.5 Dataset

Capítulo 3

Resultados y Análisis

En este capítulo se presentan los experimentos realizados para abordar el problema de segmentación de texto basada en coherencia semántica. El objetivo es maximizar la coherencia interna de los segmentos mediante un modelo de lenguaje de gran tamaño (LLM). Se llevaron a cabo tres experimentos principales: (1) ajuste de la función objetivo, (2) ajuste de hiperparámetros de las metaheurísticas y (3) evaluación comparativa de las metaheurísticas sobre instancias de prueba. Se utilizaron tres instancias de validación (con $n = 20, 15, 20$ elementos) para el ajuste, y cinco instancias de prueba (todas con $n = 20$ elementos) para la evaluación final. La métrica principal fue el puntaje F_1 entre los cortes predichos y los reales. Previamente, se precomputaron los valores de coherencia de todos los segmentos de las instancias de validación para acelerar las evaluaciones.

3.1 Experimento 1: Ajuste de la función objetivo

3.1.1. Objetivo

Determinar los valores óptimos de los parámetros de la función objetivo: la coherencia unitaria γ (valor asignado a segmentos de un solo elemento) y el factor de penalización α por cada segmento.

3.1.2. Función objetivo propuesta

La función de fitness para una partición $S = \{s_1, s_2, \dots, s_k\}$ de la secuencia de n elementos se define como:

$$f(S) = \sum_{s \in S} \max(\text{coherencia}(s), \gamma) - \alpha \cdot |S|$$

donde $\text{coherencia}(s)$ es la cohesión semántica del segmento s evaluada por el LLM (en un rango aproximado de 0 a 1), γ es un valor mínimo para segmentos de un solo elemento y α controla la penalización por número de segmentos.

3.1.3. Metodología

Se realizó una búsqueda en cuadrícula variando $\gamma \in [0,0,1,0]$ con paso 0,1 y $\alpha \in [0,0,1,0]$ con paso 0,1. Para cada combinación se ejecutó un algoritmo de programación dinámica (solución exacta) sobre las tres instancias de validación, calculando el F_1 respecto a los cortes reales. Se seleccionaron los parámetros que maximizaron el F_1 medio.

3.1.4. Resultados esperados

- **Valores altos de γ (coherencia unitaria alta):** se esperaba que provocaran una sobre-segmentación, ya que los segmentos de un solo elemento serían considerados muy coherentes, lo que aumentaría el número de cortes.
- **Penalización baja (α pequeño):** se esperaba que favoreciera particiones con muchos segmentos, mientras que penalizaciones altas (α grande) desalentarían la creación de segmentos, llevando a particiones más gruesas.
- **Mejor combinación:** se anticipaba un equilibrio intermedio, probablemente con γ moderado y α no extremo.

3.1.5. Resultados obtenidos

Se analizaron tres casos según el nivel de segmentación real de cada instancia.

Caso 1: Segmentación alta (Instancia 1, n=20) La instancia presenta una segmentación fina con muchos cortes. El mapa de calor de la Figura 3.1 muestra que el máximo F_1 se alcanza con $\gamma = 0,6$ y $\alpha = 1,0$. Penalizaciones bajas producen demasiados cortes, mientras que γ muy alto también incrementa la segmentación.

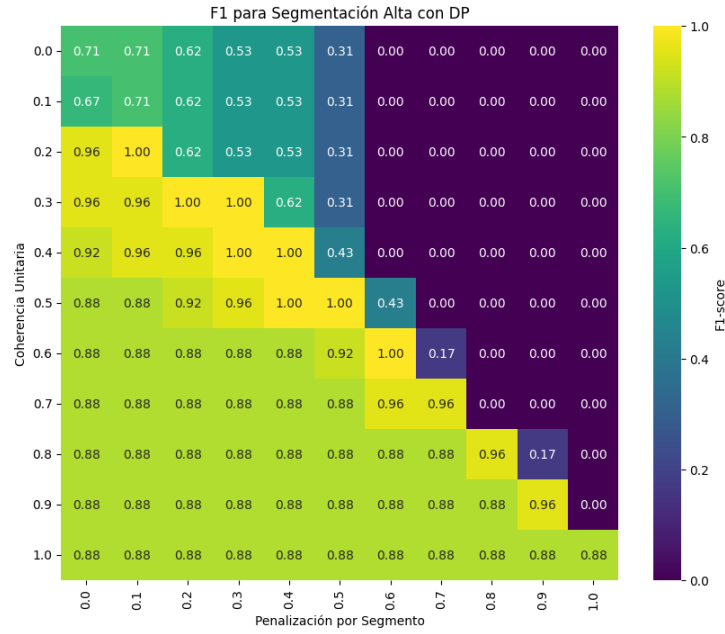


Figura 3.1: Mapa de calor del F_1 medio para la instancia 1.

Caso 2: Segmentación media (Instancia 2, n=15) La instancia tiene un número moderado de cortes. De nuevo, la combinación $\gamma = 0,6$, $\alpha = 1,0$ es la óptima (Figura 3.2). Se observa que la región de alto F_1 es más amplia que en el caso anterior.

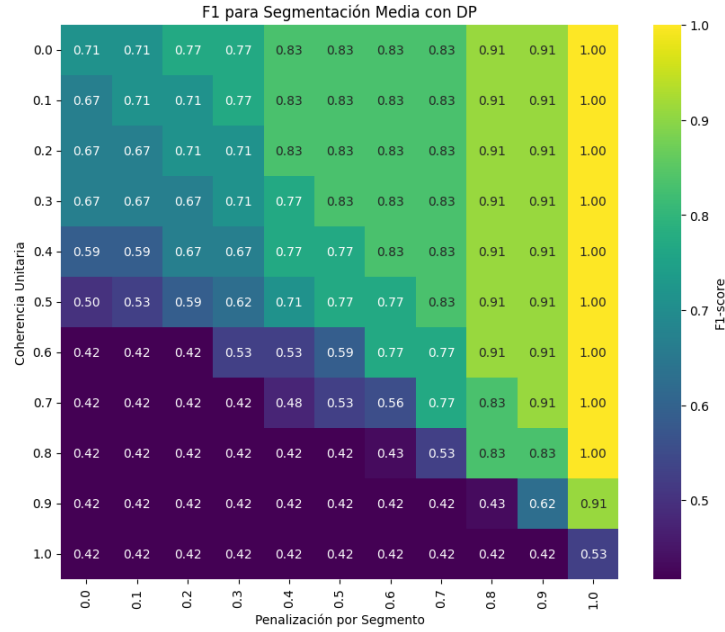


Figura 3.2: Mapa de calor del F_1 medio para la instancia 2.

Caso 3: Segmentación baja (Instancia 3, n=20) La instancia tiene pocos cortes reales. El óptimo se mantiene en $\gamma = 0,6$, $\alpha = 1,0$ (Figura 3.3). Los valores de α cercanos a 0 producen una segmentación excesiva, penalizando el F_1 .

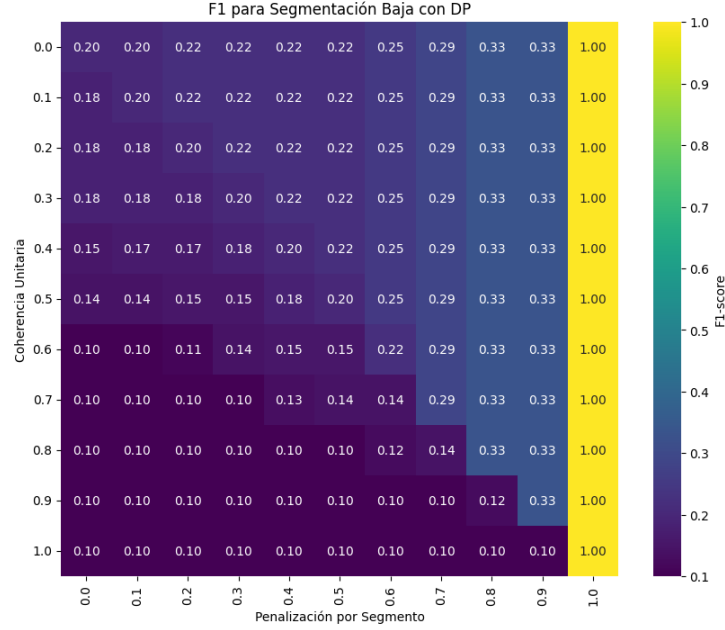


Figura 3.3: Mapa de calor del F_1 medio para la instancia 3.

3.1.6. Selección final de parámetros

La Tabla 3.1 resume los mejores parámetros encontrados para cada instancia. En todos los casos la combinación óptima fue $\gamma = 0,6$ y $\alpha = 1,0$. A pesar de que $\alpha = 1,0$ hace que la función de fitness sea negativa, resultó ser la mejor opción para equilibrar la coherencia y el número de segmentos. Por lo tanto, se fijaron $\gamma = 0,6$ y $\alpha = 1,0$ para todos los experimentos posteriores.

Cuadro 3.1: Mejores parámetros de la función objetivo por instancia de validación

Instancia	γ óptimo	α óptimo
1 (n=20, segmentación alta)	0.6	1.0
2 (n=15, segmentación media)	0.6	1.0
3 (n=20, segmentación baja)	0.6	1.0

3.2 Experimento 2: Ajuste de las metaheurísticas

3.2.1. Objetivo

Encontrar la configuración de hiperparámetros que maximice el rendimiento de cada metaheurística (Ascenso de colina, Recocido simulado, Algoritmo genético y Enjambre de partículas) sobre el problema de segmentación.

3.2.2. Metodología

Para cada algoritmo se muestrearon aleatoriamente 10 combinaciones de hiperparámetros dentro de rangos predefinidos. Cada configuración se evaluó sobre la primera instancia de validación ($n = 20$) con 3 repeticiones. Se midieron el F_1 medio y el fitness real medio (coherencia total menos penalización). Posteriormente se seleccionó la mejor configuración (mayor F_1 medio) y se registró su curva de convergencia.

3.2.3. Ascenso de colina (Hill Climbing)

- **Rango explorado:** `stagnation_limit` $\in \{50, 100, 150, 200\}$, `max_evaluations` $\in \{500, 1000, 5000\}$, `neighborhood_size` $\in \{1, 3, 5\}$.
- **Muestreo y resultados:** Tabla 3.2.
- **Mejor configuración:** `{stagnation_limit=150, max_evaluations=1000, neighborhood_size=5}` con F_1 medio = 1.0000.
- **Convergencia:** Figura 3.4.

Cuadro 3.2: Resultados del muestreo aleatorio para Hill Climbing

Parámetros	F1 medio	Fitness medio
<code>{stagnation_limit=150, max_eval=500, neigh=5}</code>	0.8571	-0.3667
<code>{stagnation_limit=50, max_eval=5000, neigh=1}</code>	0.7143	-0.5333
<code>{stagnation_limit=150, max_eval=1000, neigh=5}</code>	1.0000	-0.2000
<code>{stagnation_limit=200, max_eval=500, neigh=3}</code>	1.0000	-0.2000
<code>{stagnation_limit=200, max_eval=1000, neigh=1}</code>	0.8571	-0.3667
<code>{stagnation_limit=150, max_eval=500, neigh=3}</code>	1.0000	-0.2000
<code>{stagnation_limit=50, max_eval=500, neigh=1}</code>	1.0000	-0.2000
<code>{stagnation_limit=100, max_eval=1000, neigh=3}</code>	1.0000	-0.2000
<code>{stagnation_limit=50, max_eval=500, neigh=5}</code>	0.6190	-0.5333
<code>{stagnation_limit=50, max_eval=1000, neigh=5}</code>	1.0000	-0.2000

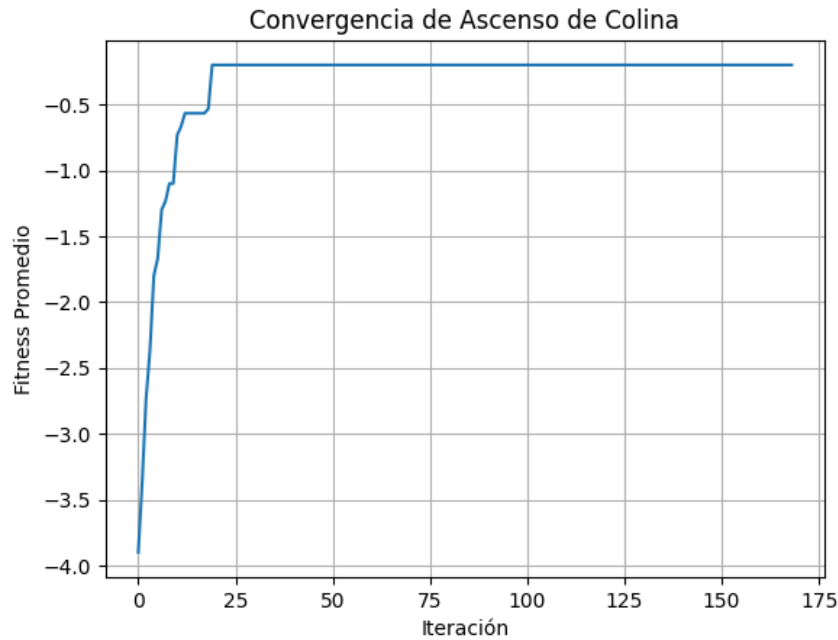


Figura 3.4: Convergencia del mejor fitness para la mejor configuración de Hill Climbing.

3.2.4. Recocido simulado (Simulated Annealing)

- **Rango explorado:** $T_0 \in \{10, 50, 100\}$, $\alpha \in \{0,8, 0,9, 0,95, 0,99\}$, $\text{iters_per_temp} \in \{5, 10, 20, 50\}$, $\text{max_evaluations} \in \{1000, 2000, 5000\}$, $T_{\min}=0.001$ fijo.
- **Muestreo y resultados:** Tabla 3.3.
- **Mejor configuración:** $\{T_0=50.0, \alpha=0.9, \text{iters_per_temp}=5, \text{max_evaluations}=2000\}$ con F_1 medio = 0.8201.
- **Convergencia:** Figura 3.5.

Cuadro 3.3: Resultados del muestreo aleatorio para Simulated Annealing

Parámetros	F1 medio	Fitness medio
{T0=50, alpha=0.9, iters=20, max_eval=5000}	0.3810	-0.7000
{T0=50, alpha=0.9, iters=20, max_eval=2000}	0.5820	-0.6667
{T0=10, alpha=0.95, iters=20, max_eval=1000}	0.4762	-0.7000
{T0=10, alpha=0.99, iters=20, max_eval=5000}	0.4762	-0.7000
{T0=10, alpha=0.8, iters=20, max_eval=5000}	0.4762	-0.7000
{T0=50, alpha=0.8, iters=5, max_eval=2000}	0.6349	-0.6000
{T0=100, alpha=0.9, iters=20, max_eval=1000}	0.4405	-0.7667
{T0=50, alpha=0.9, iters=5, max_eval=2000}	0.8201	-0.5333
{T0=100, alpha=0.99, iters=20, max_eval=5000}	0.4868	-0.6667
{T0=100, alpha=0.9, iters=10, max_eval=5000}	0.4762	-0.7000

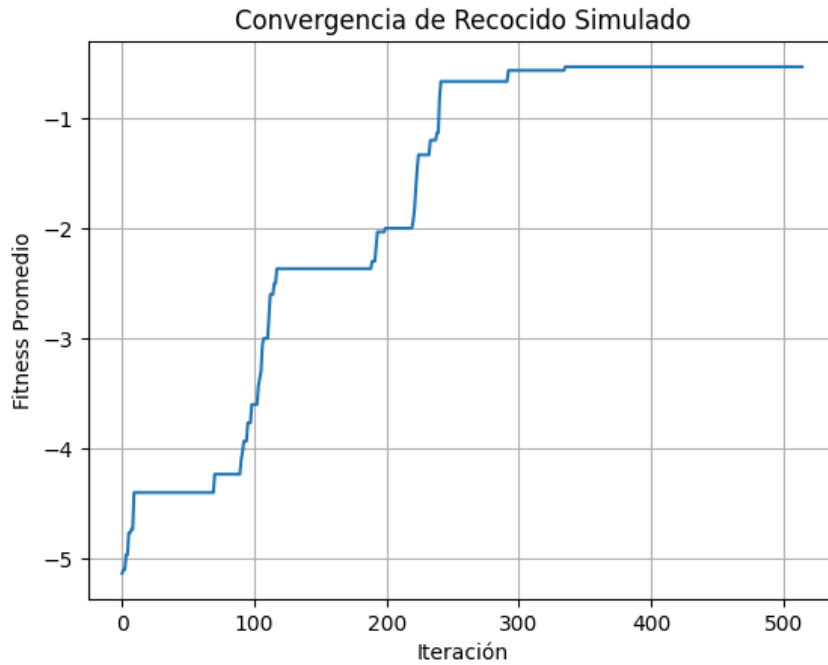


Figura 3.5: Convergencia del mejor fitness para la mejor configuración de Simulated Annealing.

3.2.5. Algoritmo genético (Genetic Algorithm)

- **Rango explorado:** `population_size` $\in \{10, 20, 50\}$, `mutation_rate` $\in \{0, 01, 0, 05, 0, 1\}$, `elitism` $\in \{1, 3, 5\}$, `max_generations` $\in \{20, 50, 100\}$, `max_evaluations` $\in \{1000, 5000\}$.
- **Muestreo y resultados:** Tabla 3.4.

- **Mejor configuración:** {population_size=50, mutation_rate=0.05, elitism=1, max_generations=100, max_evaluations=5000} con F_1 medio = 1.0000.
- **Convergencia:** Figura 3.6.

Cuadro 3.4: Resultados del muestreo aleatorio para Algoritmo Genético

Parámetros	F1 medio
{pop=50, mut=0.05, elit=1, max_gen=100, max_eval=5000}	1.0000
{pop=10, mut=0.1, elit=5, max_gen=50, max_eval=1000}	0.7778
{pop=10, mut=0.05, elit=5, max_gen=50, max_eval=5000}	0.6778
{pop=50, mut=0.05, elit=1, max_gen=20, max_eval=5000}	1.0000
{pop=50, mut=0.1, elit=3, max_gen=100, max_eval=5000}	0.7619
{pop=20, mut=0.01, elit=3, max_gen=100, max_eval=5000}	0.5464
{pop=50, mut=0.05, elit=3, max_gen=100, max_eval=1000}	1.0000
{pop=20, mut=0.1, elit=5, max_gen=20, max_eval=5000}	0.8333
{pop=20, mut=0.1, elit=5, max_gen=50, max_eval=1000}	0.7619
{pop=10, mut=0.05, elit=3, max_gen=20, max_eval=5000}	0.6086

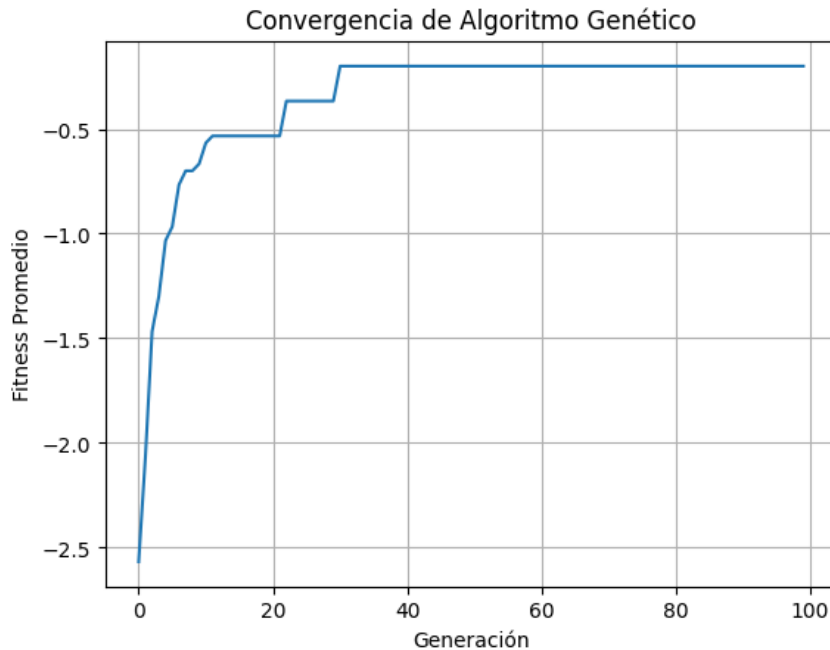


Figura 3.6: Convergencia del fitness promedio para la mejor configuración del Algoritmo Genético.

3.2.6. Enjambre de partículas (Particle Swarm Optimization)

- **Rango explorado:** `population_size` $\in \{10, 20, 50\}$, `w` $\in \{0, 5, 0, 7, 0, 9\}$, `c1` $\in \{1, 0, 1, 5, 2, 0, 2, 5\}$, `c2` $\in \{1, 0, 1, 5, 2, 0, 2, 5\}$, `max_generations` $\in \{20, 50, 100\}$, `max_evaluations` $\in \{1000, 5000\}$, `v_max` $\in \{2, 0, 5, 0, 10, 0\}$.
- **Muestreo y resultados:** Tabla 3.5.
- **Mejor configuración:** `{population_size=50, w=0.7, c1=2.0, c2=2.0, max_generations=100, max_evaluations=5000, v_max=5.0}` con F_1 medio = 1.0000.
- **Convergencia:** Figura 3.7.

Cuadro 3.5: Resultados del muestreo aleatorio para PSO

Parámetros	F1 medio
{pop=10, w=0.9, c1=2.5, c2=2.0, max_gen=100, max_eval=5000, v_max=2.0}	0.9195
{pop=50, w=0.7, c1=1.0, c2=2.0, max_gen=20, max_eval=1000, v_max=10.0}	0.9117
{pop=10, w=0.5, c1=2.0, c2=1.5, max_gen=50, max_eval=5000, v_max=2.0}	0.8012
{pop=50, w=0.9, c1=1.0, c2=2.5, max_gen=50, max_eval=1000, v_max=2.0}	0.8779
{pop=50, w=0.7, c1=1.0, c2=2.0, max_gen=50, max_eval=5000, v_max=2.0}	0.9405
{pop=50, w=0.7, c1=2.0, c2=2.0, max_gen=100, max_eval=5000, v_max=5.0}	1.0000
{pop=20, w=0.7, c1=1.5, c2=1.0, max_gen=50, max_eval=1000, v_max=5.0}	0.8824
{pop=10, w=0.7, c1=1.0, c2=2.0, max_gen=50, max_eval=1000, v_max=5.0}	0.8453
{pop=50, w=0.5, c1=1.5, c2=2.5, max_gen=20, max_eval=1000, v_max=5.0}	0.8934
{pop=10, w=0.5, c1=1.5, c2=1.5, max_gen=50, max_eval=1000, v_max=2.0}	0.8196

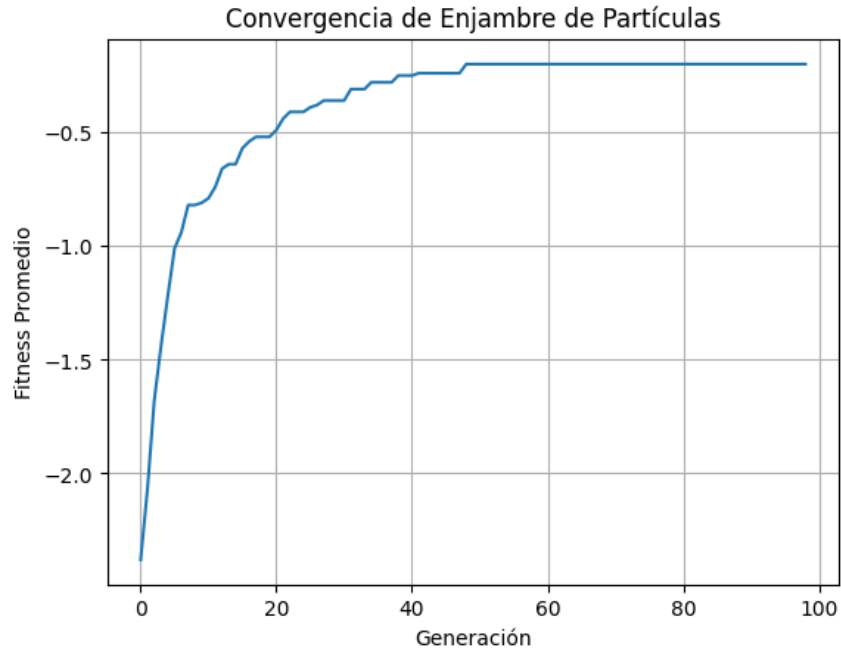


Figura 3.7: Convergencia del mejor fitness para la mejor configuración de PSO.

3.3 Experimento 3: Evaluación de las metaheurísticas

3.3.1. Objetivo

Comparar el rendimiento de las cuatro metaheurísticas, cada una con su mejor configuración hallada en el Experimento 2, sobre un conjunto de cinco instancias de prueba no vistas durante el ajuste (índices 4 a 8 del conjunto de datos, todas con $n = 20$ elementos). La métrica principal fue el F_1 medio.

3.3.2. Metodología

Para cada metaheurística se ejecutó la mejor configuración obtenida en el Experimento 2 sobre cada una de las cinco instancias de prueba, registrando el F_1 y el fitness real. Se reportan la media, desviación estándar, mínimo, máximo y el número promedio de evaluaciones.

3.3.3. Resultados esperados

- Las metaheurísticas poblacionales (Algoritmo genético y PSO) deberían superar a las de búsqueda local (Hill Climbing y Simulated Annealing) debido a su capacidad para explorar mejor el espacio de búsqueda combinatorio.
- Se esperaba que PSO, al mantener una memoria de las mejores posiciones, fuera especialmente efectivo.

- El recocido simulado podría mostrar una alta variabilidad debido a su dependencia de la temperatura inicial y el enfriamiento.

3.3.4. Resultados obtenidos

La Tabla 3.6 resume los resultados. El Enjambre de partículas (PSO) alcanzó el mejor F_1 medio (0.9025), seguido muy de cerca por el Algoritmo genético (0.8814). El ascenso de colina obtuvo un rendimiento intermedio (0.7162) y el recocido simulado mostró la mayor variabilidad y el peor valor medio (0.5447).

Cuadro 3.6: Resultados comparativos sobre las cinco instancias de prueba

Metaheurística	F1 medio	F1 std	F1 min	F1 max	Evaluaciones
Particle Swarm Optimization	0.9025	0.0000	0.9025	0.9025	1000
Genetic Algorithm	0.8814	0.0000	0.8814	0.8814	1037
Hill Climbing	0.7162	0.0000	0.7162	0.7162	496
Simulated Annealing	0.5447	0.1316	0.3578	0.7348	246

La Figura 3.8 muestra las curvas de convergencia promediadas de las cuatro metaheurísticas durante sus ejecuciones.

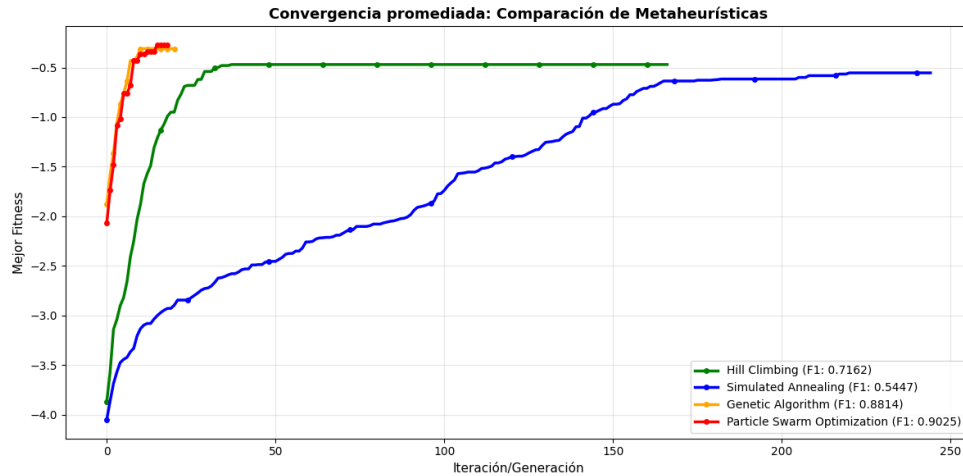


Figura 3.8: Comparación de las curvas de convergencia (mejor fitness) de las cuatro metaheurísticas sobre las instancias de prueba.

3.3.5. Conclusiones del experimento 3

El Enjambre de partículas resultó ser la metaheurística más efectiva para el problema de segmentación basada en coherencia semántica, logrando un F_1 medio de 0.9025. Su capacidad de equilibrar exploración y explotación, junto con la memoria de las mejores posiciones individuales y globales, le permitió encontrar particiones de alta calidad. El

Algoritmo genético también mostró un rendimiento excelente, aunque ligeramente inferior. Las metaheurísticas de búsqueda local se vieron limitadas por la naturaleza multimodal del espacio de búsqueda. Por lo tanto, se selecciona PSO con la configuración encontrada como la estrategia final para el sistema.

Capítulo 4

Conclusiones

En este trabajo se ha abordado el problema de segmentación óptima de contenido guiada por coherencia semántica, empleando técnicas clásicas de optimización y un LLM como evaluador. Los resultados muestran que:

- La programación dinámica es viable como referencia exacta en instancias pequeñas, pero su complejidad cuadrática en número de evaluaciones limita su aplicación práctica cuando el evaluador es un LLM.
- Las metaheurísticas, en particular el Simulated Annealing con caché, logran aproximar el óptimo con un número controlado de consultas al modelo de lenguaje.
- La integración de un modelo de lenguaje como función de evaluación semántica es factible y aporta una noción de coherencia más profunda que las métricas puramente léxicas, siempre que se gestione adecuadamente el coste asociado.
- Estrategias como el uso de sustitutos rápidos (embeddings) y el almacenamiento en caché resultan esenciales para viabilizar la optimización en escenarios realistas.

Se ha cumplido el doble objetivo de aplicar técnicas de búsqueda y optimización de la asignatura y de incorporar un LLM como componente funcional desacoplado. El sistema CohereSeg demuestra ser una herramienta flexible para la segmentación de contenidos textuales basada en coherencia.

Capítulo 5

Recomendaciones